

Lab 8: Operator Overloading

1. Lab Overview

This lab is related to operator overloading in C++ using a single, and focused class called SmartArray. SmartArray class will support the operators ++, --, +, -, *, +=, ==, <<, and [] that are not supported by built-in arrays in C++.

Learning Objectives

#	Learning Objective
1	Overload unary operators in both prefix and postfix form.
2	Overload binary arithmetic, compound-assignment, and relational operators.
3	Decide between member-function and friend-function overloads.
4	Distinguish return-by-value from return-by-reference and use *this correctly.
5	Manage dynamic memory inside operator implementations.
6	Overload the subscript operator [] to support both reading and writing.
7	Provide a friend stream-insertion operator << for clean object printing.

2. Introduction

Operator overloading lets you redefine what built-in operators (+, -, *, ++, ==, [], <<, and many others) mean when their operands are objects of a class you wrote. The compiler picks the correct version based on the static types of the operands, which is why operator overloading is called compile-time polymorphism.

General syntax

```
return_type operator<symbol>( parameter_list )
{
    // implementation
}
```

What can be overloaded?

Almost every C++ operator can be overloaded. The exceptions you must remember are:

- :: (scope resolution)
- . (member access)
- ?: (ternary conditional)
- sizeof and typeid

Member function vs. friend function

An overload written as a member function receives the left operand implicitly through the this pointer. A friend (or non-member) overload receives every operand explicitly as a parameter. As a rule of thumb, when the left operand is a non-class type — for example an ostream in cout << obj — you cannot use a member function and must use a friend or free function.

3. The Starting Point

Before any operator can be written, we need a class that holds something. SmartArray wraps a raw `int*` buffer plus its size. We give it a single constructor and a `show()` helper so we can see what is inside while we build.

```
#include <iostream>
using namespace std;

class SmartArray {
    int* array;
    int  sz;
public:
    SmartArray(int* ptr, int s) {
        array = ptr;
        sz    = s;
    }

    void show() {
        cout << "[";
        for (int i = 0; i < sz; i++) {
            cout << *(array + i);
            if (i < sz - 1) cout << ", ";
        }
        cout << "]" << endl;
    }
};

int main() {
    const int MAX = 3;
    int x[MAX] = {1, 2, 3};

    SmartArray s1(x, MAX);
    s1.show();
    return 0;
}
```

Output:

```
[1, 2, 3]
```

Why does the constructor take `int* ptr` instead of copying values?

The class is a thin wrapper at this stage. It stores the address of an existing array and its size. That keeps the example focused on operator overloading rather than on memory management. Later, when operators like `+` need to produce a brand new `SmartArray`, we will have to allocate memory ourselves with `new`.

4. Unary Operators

A unary operator acts on a single operand. We will overload `++` and `--` so that incrementing or decrementing a `SmartArray` adds or subtracts one from every element in the buffer. Because `++` and `-` exist in two flavours - prefix (`++a`) and postfix (`a++`) - we need two different overloads with different signatures.

4.1 Pre-increment `++a`

The prefix form is overloaded as `operator++()` with no parameters. We mutate the object in place and, in the simplest possible version below, return nothing.

```
void operator++() {
    for (int i = 0; i < sz; i++) {
        ++*(array + i);
    }
}
```

Test driver:

```
int x[3] = {1, 2, 3};
SmartArray s1(x, 3);

s1.show();    // before
++s1;        // pre-increment
s1.show();    // after
```

Output:

```
[1, 2, 3]
[2, 3, 4]
```

What is happening behind the scenes?

When the compiler sees `++s1`, it rewrites the expression as `s1.operator++()`. The call has no explicit argument because the only operand, `s1`, is the implicit `this` object. Inside the function the loop dereferences (`array + i`) and increments the integer that lives there. Remember, we are not calling the operator using dot operator.

4.2 Post-increment `a++` and the dummy int parameter

Prefix and postfix have the same operator symbol, so C++ needs a way to tell the two overloads apart. The convention is that postfix takes an unused int parameter. The compiler quietly passes 0 for that parameter; you never use it. Postfix must also return the OLD state of the object, because in expressions like `cout << s1++` the user expects to see the value before the increment.

```
SmartArray operator++(int) {    // dummy int -> postfix
    SmartArray old = *this;    // copy current state
    for (int i = 0; i < sz; i++) {
        ++*(array + i);        // mutate this
    }
    return old;                // return the saved copy
}
```

Test driver:

```
int x[3] = {1, 2, 3};
SmartArray s1(x, 3);

s1.show();    // [1, 2, 3]
SmartArray snap = s1++; // postfix: snap = old, s1 increments
cout << "snap : "; snap.show();
cout << "s1   : "; s1.show();
```

Output:

```
[1, 2, 3]
snap : [1, 2, 3]
s1   : [2, 3, 4]
```

Why does pre return void or a reference, but post return by value?

Pre-increment can return a reference to the object that was just modified, so chained code such as `++(++s1)` keeps working. Post-increment cannot return the object itself, because the object has already advanced. It must return a separate snapshot taken before the change - that is what `SmartArray old = *this` does. Returning that snapshot by value forces the compiler to make a copy, leaving the snapshot safe even after the original object continues to live and change.

4.3 Pre-decrement `--a` and Post-decrement `a--`

Decrement mirrors increment exactly. The only thing that changes is the operator symbol and the direction of the change.

```
void operator--() {                // pre
    for (int i = 0; i < sz; i++)
        --*(array + i);
}

SmartArray operator--(int) {        // post
    SmartArray old = *this;
    for (int i = 0; i < sz; i++)
        --*(array + i);
    return old;
}
```

Test driver:

```
int x[3] = {5, 5, 5};
SmartArray s1(x, 3);

s1.show();
--s1;
s1.show();
SmartArray snap = s1--;
cout << "snap : "; snap.show();
cout << "s1   : "; s1.show();
```

Output:

```
[5, 5, 5]
[4, 4, 4]
snap : [4, 4, 4]
s1    : [3, 3, 3]
```

5. Binary Operators

A binary operator takes two operands. As a member function, the left operand is the implicit `this` object and the right operand is the explicit parameter. We will add five binary operators in this section: `+`, `-`, `*`, `+=`, and `==`.

5.1 `operator+` (Element-wise addition)

Adding two `SmartArrays` should produce a third `SmartArray` whose *i*-th element is the sum of the *i*-th elements of the operands. The result is a brand new object, so we must allocate a new buffer with `new int[sz]`.

```

SmartArray operator+(SmartArray s) {           // s = right operand
    int* temp = new int[sz];                  // allocate new buffer
    for (int i = 0; i < sz; i++) {
        *(temp + i) = *(array + i) + *(s.array + i);
    }
    return SmartArray(temp, sz);              // return new object
}

```

Test driver:

```

int x[3] = {1, 2, 3};
int y[3] = {7, 8, 9};
SmartArray s1(x, 3), s2(y, 3);

SmartArray s3 = s1 + s2;
s3.show();
s1.show();
s2.show();

```

Output:

```

[8, 10, 12]
[1, 2, 3]
[7, 8, 9]

```

Object as ARGUMENT - pass by value

The parameter SmartArray s receives a COPY of s2. Inside operator+, the line `*(s.array + i)` reads from that copy. Because s holds the same pointer as s2, both objects look at the same int buffer in memory - which is fine here because we only read it. Pass-by-value is acceptable for small classes, but for larger objects, it is more idiomatic to write `const SmartArray& s` to avoid the copy and to declare the function const-friendly.

Object RETURNED - return by value

The expression `return SmartArray(temp, sz)` constructs a temporary SmartArray on the spot. The compiler initialises s3 from that temporary. The new `int[sz]` buffer that temp points to lives on the heap, so it survives even though the temporary itself disappears at the end of the statement.

Why new ?

Stack arrays such as `int temp[sz]` would die at the `}` that closes operator+. Returning a SmartArray that pointed at dead memory would crash. By writing `new int[sz]` we put the buffer on the heap, where it stays alive until somebody calls `delete[]`. In a production class you would couple this with a destructor that calls `delete[]` on the buffer when the object dies.

5.2 operator- (Element-wise subtraction)

Subtraction is the same pattern as addition, with - in place of +.

```

SmartArray operator-(SmartArray s) {
    int* temp = new int[sz];
    for (int i = 0; i < sz; i++)
        *(temp + i) = *(array + i) - *(s.array + i);
}

```

```
    return SmartArray(temp, sz);  
}
```

Test driver:

```
int x[3] = {10, 20, 30};  
int y[3] = {1, 2, 3};  
SmartArray a(x, 3), b(y, 3);  
  
SmartArray c = a - b;  
c.show();
```

Output:

```
[9, 18, 27]
```

5.3 operator* (The SAME symbol, TWO different overloads)

A SmartArray multiplication is meaningful in two different ways. We can multiply two SmartArrays element-wise, or we can multiply every element by an integer scalar. Both share the symbol *, but they have different parameter lists, so the compiler can tell them apart - this is function overloading applied to operators.

```
// (a) array * array - element-wise product  
SmartArray operator*(SmartArray s) {  
    int* temp = new int[sz];  
    for (int i = 0; i < sz; i++)  
        temp[i] = array[i] * s.array[i];  
    return SmartArray(temp, sz);  
}  
// (b) array * int - scalar multiplication  
SmartArray operator*(int x) {  
    int* temp = new int[sz];  
    for (int i = 0; i < sz; i++)  
        temp[i] = *(array + i) * x;  
    return SmartArray(temp, sz);  
}
```

Test driver:

```
int x[3] = {1, 2, 3};  
int y[3] = {4, 5, 6};  
SmartArray a(x, 3), b(y, 3);  
  
SmartArray prod = a * b;    // overload (a)  
SmartArray scl = a * 10;    // overload (b)  
prod.show();  
scl.show();
```

Output:

```
[4, 10, 18]  
[10, 20, 30]
```

Asymmetry of $a * 10$ vs. $10 * a$

Because `operator*(int)` is a member function, the left operand has to be a SmartArray. The expression `10 * a` would not compile: 10 is not a SmartArray, so it has no `operator*` member to call. To support

10 * a you would write a free (or friend) function operator*(int x, SmartArray s). The lesson is that member-function overloads are inherently asymmetric, while free-function overloads are symmetric.

5.4 operator+= (Return *this by reference)

Compound assignment mutates the left operand and is conventionally chainable: (a += b) += c. To make that work, the operator must return a reference to *this rather than a copy.

```
SmartArray& operator+=(SmartArray s) {
    for (int i = 0; i < sz; i++)
        *(array + i) += *(s.array + i);
    return *this;           // reference to the modified object
}
```

Test driver:

```
int x[3] = {1, 2, 3};
int y[3] = {7, 8, 9};
SmartArray a(x, 3), b(y, 3);
a += b;
a.show();
```

Output:

```
[8, 10, 12]
```

What is *this ?

Inside any non-static member function the compiler hands you a hidden pointer named this that points to the object the function was called on. Writing *this dereferences that pointer to refer to the object itself. Returning *this by reference (SmartArray&) means the caller gets the actual object back, not a copy, which is exactly what is needed for chaining.

5.5 operator== (A relational operator returns bool)

Equality compares two SmartArrays element by element. Unlike the arithmetic operators, the result type here is bool, not SmartArray.

```
bool operator==(SmartArray s) {
    if (sz != s.sz) return false;
    for (int i = 0; i < sz; i++)
        if (*(array + i) != *(s.array + i)) return false;
    return true;
}
```

Test driver:

```
int x[3] = {1, 2, 3};
int y[3] = {1, 2, 3};
int z[3] = {9, 9, 9};
SmartArray a(x, 3), b(y, 3), c(z, 3);

cout << (a == b ? "a == b" : "a != b") << endl;
cout << (a == c ? "a == c" : "a != c") << endl;
```

Output:

```
a == b
a != c
```

5.6 Binary operators with three operands (Left-to-right chaining)

A binary operator literally accepts two operands at a time. So how does the language handle the everyday expression $a + b + c$, which mentions three? The answer is associativity: $+$ is left-associative, so the compiler parses the expression as $(a + b) + c$. It evaluates the left pair first, takes the temporary result, and uses that as the left operand of the second $+$.

```
int x[3] = {1, 1, 1};
int y[3] = {2, 2, 2};
int z[3] = {3, 3, 3};
SmartArray a(x, 3), b(y, 3), c(z, 3);

SmartArray result = a + b + c;    // ((a + b) + c)
result.show();
```

Output:

```
[6, 6, 6]
```

Step by step the compiler does:

- Call `(a).operator+(b)` - produces an unnamed temporary `SmartArray` with values `[3, 3, 3]`.
- Call `temporary.operator+(c)` - produces another temporary with values `[6, 6, 6]`.
- Construct `result` from that final temporary.

So a single binary operator can transparently participate in expressions of any length. The same logic applies to relational chains like `a == b == c`, although those rarely make sense semantically.

6. Stream Insertion operator<<

Up until now we have called `s1.show()` to print a `SmartArray`. We would like to write `cout << s1` instead. The left operand of `<<` is an ostream object, not a `SmartArray`, so we cannot use a member-function overload. Instead we declare a free function and grant it friend access so that it can read the private fields of `SmartArray`.

Inside the class

```
friend ostream& operator<<(ostream& out, const SmartArray& obj);
```

Outside the class

```
ostream& operator<<(ostream& out, const SmartArray& obj) {
    out << "[";
    for (int i = 0; i < obj.sz; i++) {
        out << obj.array[i];
        if (i < obj.sz - 1) out << ", ";
    }
    out << "]";
    return out;    // return the stream so chaining works
}
```


Test driver:

```
int x[3] = {1, 2, 3};
SmartArray s1(x, 3);

cout << "s1 = " << s1 << endl;
```

Output:

```
s1 = [1, 2, 3]
```

Why return ostream& ?

The expression `cout << "s1 = " << s1 << endl` is parsed as `((cout << "s1 = ") << s1) << endl`. After our overload runs, the result of the sub-expression must again be the same `cout` so that the next `<<` has something to write to. Returning out by reference does exactly that.

Why const SmartArray& ?

Reading a value to print it does not require modifying the object, so we accept it by const reference. `const` protects callers, and the reference avoids copying.

7. Subscript Operator (operator[])

The subscript operator `[]` makes a `SmartArray` feel like a built-in array. We want both reading (`int v = s1[2]`) and writing (`s1[2] = 100`) to work. That single requirement drives every design decision in this section.

7.1 The minimal version

```
int& operator[](int x) {
    if (x < 0 || x >= sz)
        exit(1);
    return array[x];
}
```

Test driver:

```
int x[3] = {1, 2, 3};
SmartArray s1(x, 3);

cout << s1[1] << endl;    // read
s1[2] = 100;              // write
cout << s1 << endl;
```

Output:

```
2
[1, 2, 100]
```

Why does operator[] return a REFERENCE?

When the expression `s1[2] = 100` runs, the compiler first evaluates `s1[2]`. If our overload returned `int` by value, that would be a temporary, and you cannot assign to a temporary and the line would not

compile. By returning `int&` we hand back the actual integer slot inside the buffer; assigning to it stores the value where it belongs.

7.2 Const correctness - add a const overload

If we have a `const SmartArray`, calling the overload above is illegal - it is a non-const member function. The conventional fix is to provide two overloads: a non-const version that returns `int&` (for read+write) and a const version that returns `const int&` (read-only).

```
int& operator[](int x) {
    if (x < 0 || x >= sz) exit(1);
    return array[x];
}

const int& operator[](int x) const {
    if (x < 0 || x >= sz) exit(1);
    return array[x];
}
```

7.3 How [] is dispatched

The compiler looks at the constness of the object on the left of the brackets:

- If the object is non-const, the non-const overload is chosen, returning `int&` - reading and writing are both fine.
- If the object is const, the const overload is chosen, returning `const int&` - reading works, writing is rejected by the compiler.

7.4 Bounds checking and error reporting

Calling `exit(1)` on an out-of-range index is forceful because it terminates the program. In professional code, you would normally throw an exception, for example `std::out_of_range`, so that callers can recover. The current `SmartArray` uses `exit(1)` only to keep the focus on operator overloading.

```
int& operator[](int x) {
    if (x < 0 || x >= sz)
        throw std::out_of_range("SmartArray index out of range");
    return array[x];
}
```

8. Putting It All Together

Below is the complete `SmartArray` class with every operator we have built. Read it as a single unit now that each piece has been justified individually.

```
#include <iostream>
using namespace std;

class SmartArray {
    int* array;
    int  sz;
public:
```

```

SmartArray(int* ptr, int s) : array(ptr), sz(s) {}

// -- diagnostics --
void show() const {
    cout << "[";
    for (int i = 0; i < sz; i++) {
        cout << *(array + i);
        if (i < sz - 1) cout << ", ";
    }
    cout << "]" << endl;
}
int size() const { return sz; }

// -- unary --
void operator++() { for (int i=0;i<sz;i++) ++*(array+i); }
void operator--() { for (int i=0;i<sz;i++) --*(array+i); }
SmartArray operator++(int) {
    SmartArray old = *this;
    for (int i=0;i<sz;i++) ++*(array+i);
    return old;
}
SmartArray operator--(int) {
    SmartArray old = *this;
    for (int i=0;i<sz;i++) --*(array+i);
    return old;
}

// -- binary arithmetic --
SmartArray operator+(SmartArray s) {
    int* t = new int[sz];
    for (int i=0;i<sz;i++) t[i] = array[i] + s.array[i];
    return SmartArray(t, sz);
}
SmartArray operator-(SmartArray s) {
    int* t = new int[sz];
    for (int i=0;i<sz;i++) t[i] = array[i] - s.array[i];
    return SmartArray(t, sz);
}
SmartArray operator*(SmartArray s) {
    int* t = new int[sz];
    for (int i=0;i<sz;i++) t[i] = array[i] * s.array[i];
    return SmartArray(t, sz);
}
SmartArray operator*(int x) {
    int* t = new int[sz];
    for (int i=0;i<sz;i++) t[i] = array[i] * x;
    return SmartArray(t, sz);
}

// -- compound assignment --
SmartArray& operator+=(SmartArray s) {
    for (int i=0;i<sz;i++) array[i] += s.array[i];
    return *this;
}

// -- relational --
bool operator==(SmartArray s) {
    if (sz != s.sz) return false;
    for (int i=0;i<sz;i++)
        if (array[i] != s.array[i]) return false;
}

```

```

        return true;
    }

    // -- subscript --
    int& operator[](int x) {
        if (x < 0 || x >= sz) exit(1);
        return array[x];
    }
    const int& operator[](int x) const {
        if (x < 0 || x >= sz) exit(1);
        return array[x];
    }

    // -- stream insertion (friend) --
    friend ostream& operator<<(ostream& out, const SmartArray& obj);
};

ostream& operator<<(ostream& out, const SmartArray& obj) {
    out << "[";
    for (int i = 0; i < obj.sz; i++) {
        out << obj.array[i];
        if (i < obj.sz - 1) out << ", ";
    }
    out << "]";
    return out;
}

int main() {
    const int MAX = 3;
    int x[MAX] = {1, 2, 3};
    int y[MAX] = {7, 8, 9};

    SmartArray s1(x, MAX);
    SmartArray s2(y, MAX);

    cout << "s1          : " << s1 << endl;
    ++s1;
    cout << "after ++s1     : " << s1 << endl;
    --s1;
    cout << "after --s1      : " << s1 << endl;

    SmartArray s3 = s1 + s2;
    cout << "s1 + s2         : " << s3 << endl;
    cout << "s1              : " << s1 << endl;
    cout << "s2              : " << s2 << endl;

    s1 += s2;
    cout << "after s1 += s2: " << s1 << endl;

    s1[2] = 100;
    cout << "after s1[2]=100: " << s1 << endl;
    cout << "s1[2]           : " << s1[2] << endl;

    SmartArray s4 = s1 * s2;
    cout << "s1 * s2         : " << s4 << endl;
    SmartArray s5 = s1 * 2;
    cout << "s1 * 2          : " << s5 << endl;

    cout << "s4 == s5 ? " << ((s4 == s5) ? "yes" : "no") << endl;
}

```

```
    return 0;
}
```

Output:

```
s1           : [1, 2, 3]
after ++s1   : [2, 3, 4]
after --s1   : [1, 2, 3]
s1 + s2      : [8, 10, 12]
s1           : [1, 2, 3]
s2           : [7, 8, 9]
after s1 += s2: [8, 10, 12]
after s1[2]=100: [8, 10, 100]
s1[2]        : 100
s1 * s2      : [56, 80, 900]
s1 * 2       : [16, 20, 200]
s4 == s5 ?   no
```

9. Key Points

The implicit this pointer

Every non-static member function has access to a hidden pointer named `this`. Writing `*this` gives you the object the function was called on. Operators that mutate the left operand return `*this` by reference so that callers can chain.

Object as argument

Passing a `SmartArray` by value (`SmartArray s`) makes a copy. Passing it by const reference (`const SmartArray& s`) avoids the copy and signals that the function will not modify `s`. For symmetric arithmetic operators we wrote pass-by-value because the buffer pointer is cheap to copy; for production code, const references are usually preferred.

Object returned

Arithmetic operators return brand new `SmartArray` objects by value, because each operation produces a fresh result. Compound-assignment and stream-insertion operators return references (`SmartArray&` and `ostream&`) so that they can be chained.

The new keyword

Whenever an operator must produce a buffer that outlives the function call, it allocates that buffer on the heap with `new int[sz]`. A stack array would die at the closing brace and leave the returned `SmartArray` pointing at invalid memory. In a polished class, every `new` is paired with a `delete[]` in a destructor.

Unary vs. binary, member vs. friend

Unary operators take zero explicit parameters when written as members. Binary operators take exactly one. Postfix versions of `++` and `--` take a dummy `int`. Whenever the left operand cannot be a `SmartArray` (the most common case being `cout << obj`) we must use a free or friend function instead of a member.

Subscript [] needs a reference

Returning by reference is what allows `s1[2] = 100` to compile. Pairing the non-const overload with a const overload is the standard idiom. It lets `[]` work on const `SmartArray` objects in read-only mode.

Binary chains - three operands or more

Operators are still binary even in expressions like `a + b + c`. Left-to-right associativity makes the compiler call `operator+` twice, threading temporary objects from one call into the next.